Gradient-Based Sensitivity Analysis in Large Language Models

Abstract

We formalize and implement a gradient-based sensitivity analysis for neural language models. Theoretical connections between per-parameter gradients and sensitivity to soft errors motivate a practical procedure: stream a language modeling corpus into fixed windows, compute next-token cross-entropy loss, backpropagate, and track the maximum absolute gradient observed per weight element across many batches to identify the most sensitive parameters.

1 Gradient-Based Sensitivity Framework

Certain parameters (elements of weight tensors) are more impactful than others when a soft error is introduced.

Notation. We define the following:

- $\vec{\theta} \in \mathbb{R}^N$: The parameter vector
$$\vec{\theta} = \begin{pmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_N \end{pmatrix}.$$
- x: The model input (e.g., a token sequence).
- y: The target/label for the input (e.g., the true next token).
- $f(x; \vec{\theta})$: The LLM's output when parameters are $\vec{\theta}$.
- $\mathcal{L}(f(x; \vec{\theta}), y)$: The scalar loss function (e.g., cross-entropy) measuring mismatch between prediction and target.
- δ: A small scalar perturbation magnitude.
- e_i: The i-th standard basis vector with a 1 in the i-th position and 0 elsewhere.

Sensitivity. The sensitivity of parameter θ_i is defined as

$$S_i(\delta) \;=\; \mathcal{L}\Big(f(x;\vec{\theta}+\delta e_i), y\Big) \;-\; \mathcal{L}\Big(f(x;\vec{\theta}), y\Big). \tag{1}$$

Using a first-order Taylor expansion, this can be approximated as

$$S_i(\delta) \;\approx\; \delta\, g_i, \qquad \text{where} \quad \boldsymbol{g} = \nabla_{\vec{\theta}}\mathcal{L}(f(x;\vec{\theta}), y). \tag{2}$$

Thus, the gradient entry g_i serves as a first-order proxy for sensitivity.

2 Methodology and Procedure

Corpus as a vector. The tokenized corpus is represented as

$$x = [x_0, x_1, \ldots, x_{T-1}], \qquad x_t \in \{0, 1, \ldots, V-1\},$$

where T is the total number of tokens and V is the vocabulary size.

Windowing. For each index n, we extract a window

$$w(n) = [x_n, x_{n+1}, \ldots, x_{n+L}],$$

where $L = \texttt{SEQ_LEN}$ is the input sequence length.

Inputs and labels. Each window is split into input and label sequences:

$$I(n) = [x_n, \ldots, x_{n+L-1}], \qquad Y(n) = [x_{n+1}, \ldots, x_{n+L}].$$

Batching. Stacking B such pairs yields input and label tensors

$$I \in \mathbb{N}^{B\times L}, \qquad Y \in \mathbb{N}^{B\times L}.$$

Forward pass. The model with parameters $\vec{\theta}$ produces logits

$$Z = f(I;\vec{\theta}) \in \mathbb{R}^{B\times L\times V}.$$

Loss. The cross-entropy loss averaged over batch and sequence positions is

$$\mathcal{L}(I, Y; \vec{\theta}) = \frac{1}{B(L-1)} \sum_{b=1}^{B} \sum_{t=0}^{L-2} -\log p\Big(Y_{b,t+1} \,\Big|\, I_{b,\le t}; \vec{\theta}\Big)\,.$$

Gradient. By backpropagation, the gradient of the loss with respect to the parameters is

$$\nabla_{\vec{\theta}} \mathcal{L}(I, Y; \vec{\theta}) = \left[\frac{\partial \mathcal{L}}{\partial \theta_1}, \ \frac{\partial \mathcal{L}}{\partial \theta_2}, \ \ldots, \ \frac{\partial \mathcal{L}}{\partial \theta_N} \right]^\top .$$

This gradient vector assigns one sensitivity score per parameter. The magnitude

$$\left| \frac{\partial \mathcal{L}}{\partial \theta_i} \right|$$

highlights the most sensitive elements in the model.

3 Experimental Results

The above procedure was conducted on the smallest model of GPT-2 (137m parameters). A sequence length of 1024 tokens, batch size of 16 and step size of 1100 was used to cover ∼18 million tokens (approx. 18%) of the Wikitext-103 text corpus.

Top-10 Most Sensitive Parameters

The top-10 most sensitive parameters were found to be:

Rank	Weight	Element	$\lvert\nabla\mathcal{L}\rvert$
1	transformer.wte.weight	(2488,496)	5.235e+00
2	transformer.wte.weight	(837,496)	4.481e+00
3	transformer.wte.weight	(198,496)	3.247e+00
4	transformer.wte.weight	(11,496)	2.898e+00
5	transformer.wte.weight	(34315,496)	2.669e+00
6	transformer.wte.weight	(796,496)	2.517e+00
7	transformer.wte.weight	(764,496)	2.499e+00
8	transformer.wte.weight	(13,496)	2.427e+00
9	transformer.wte.weight	(220,496)	2.242e+00
10	transformer.wte.weight	(2488,430)	2.207e+00

The Top-10 Most Sensitive Parameters (excluding the embedding layer, and ayer norm weights):

Local Rank	Global Rank	Weight	Element	$\lvert\nabla\mathcal{L}\rvert$
1	117	transformer.h.5.mlp.c_fc.weight	(393, 1866)	1.341e+00
2	123	transformer.h.5.mlp.c_fc.weight	(373, 1866)	1.319e+00
3	134	transformer.h.5.mlp.c_fc.weight	(64, 1866)	1.298e+00
4	158	transformer.h.5.mlp.c_fc.weight	(447, 1866)	1.177e+00
5	219	transformer.h.5.mlp.c_fc.weight	(326, 1866)	1.007e+00
6	242	transformer.h.5.mlp.c_fc.bias	(1866)	9.751e-01
7	243	transformer.h.5.mlp.c_fc.weight	(138, 1866)	9.741e-01
8	246	transformer.h.6.attn.c_attn.weight	(64, 2197)	9.673e-01
9	262	transformer.h.7.attn.c_attn.weight	(64, 2296)	9.316e-01
10	300	transformer.h.3.mlp.c_fc.weight	(393, 1612)	8.853e-01

Bit-Flip Trials

A single bit flip was conducted repeatedly (e.g. 5 times) on different sections of the bits (sign, exponent, mantissa bits) for different input prompts. The following prompts were taken:

1. The weather today is
2. The patient should take
3. The bank transfer amount is
4. The recommended dose for a child is
5. The evacuation order status is

The model generated 100 tokens as the output. The outputs were classified into three categories:

- **Preserved** - The meaning of the output was relatively unchanged. Condition: BERTScore_F1 > 0.87
- **Changed** - The output was intelligible, but the meaning had changed. Condition: $0.80 <$ BERTScore_F1 < 0.87
- **Gibberish** - The output had completely become a random jumble of words and symbols. Condition: BERTScore_F1 < 0.80

Top-3 Most Sensitive Element Test

The top-3 most sensitive tensor elements were tested. Out of 225 outputs:

Classification	Count	Example
Preserved	4	Input: The patient should take Clean Output: a blood test to confirm the diagnosis. The patient should be given a blood test to confirm the diagnosis. Corrupted Output: a blood test to confirm the diagnosis. The patient will be given a blood sample and a urine sample.
Changed	6	Input: The weather today is Clean Output: very good, and we're going to be able to get some good weather tomorrow. Corrupted Output: very good, and we're going to be able to get some good weather tomorrow. "We're going into the weekend and we'll be able, hopefully, to get a good start to the season." The Reds have been without a win in their last five games, with a 3-1 defeat at home to the Royals on Saturday..@RedsReds are in the midst of a 3rd straight winless start. #Reds — RedsNation (@R
Gibberish	215	Input: The evacuation order status is Clean Output: being reviewed by the Department of Homeland Security. The evacuation order was issued by the Department of Homeland Security on July 1, 2017. Corrupted Output: , , , and , , respectively , , . , „ , or , , are , , (, ,), , , etc , , in , , as , , & , , for , , with , , without , , by , , on , , under , , out , , to , , through , , over , , at , , from , , into , , back , , across , , between , , within , , around , , outside , , inside , , above

Analysis

The results indicate that the vast majority (95.6%) of outputs degraded into gibberish when a single bit flip was introduced in the top-3 most sensitive parameters, which were classified as 'Gibberish'. A smaller fraction (2.67%) produced intelligible text whose semantic meaning diverged significantly from the clean baseline, which were classified as 'Changed', while only 1.78% of generations maintained their intended meaning, which were classified as 'Preserved'. These results underscore both the extreme fragility of highly sensitive parameters to low-level perturbations and the rarity of semantically robust outputs under such faults.

4 Conclusion

We presented a gradient-based framework for estimating parameter sensitivity and a practical scan that identifies the most sensitive elements in large language models. This offers a principled way to target robustness interventions (e.g., bit-flip testing, weight clipping, or redundancy) at the most impactful parameters.

A Tokenization and Batching Procedure

The dataset is tokenized and streamed into a rolling buffer (cache). From this buffer, windows of fixed length are extracted, split into inputs and labels, and grouped into batches.

Algorithm A.1: Tokenization and Batching

Input: Tokenized corpus $\mathbf{x} = [x_0, x_1, \ldots, x_{T-1}]$, sequence length L, batch size B.
Output: Batches of inputs $I \in \mathbb{N}^{B \times L}$ and labels $Y \in \mathbb{N}^{B \times L}$.

1. Initialize an empty rolling buffer `cache`.
2. Stream tokens x_t into the buffer.
3. While the buffer has at least $L + 1$ tokens:
   (a) Take a window $\mathbf{w} = [x_n, \ldots, x_{n+L}]$.
   (b) Define inputs $\mathbf{I} = [x_n, \ldots, x_{n+L-1}]$.
   (c) Define labels $\mathbf{Y} = [x_{n+1}, \ldots, x_{n+L}]$.
   (d) Store $(\mathbf{I}, \mathbf{Y})$.
   (e) Remove consumed tokens from the buffer.
4. Group stored pairs into mini-batches of size B.

Code Illustration

```
def chunk_generator(x, L):
    cache = []
    for token in x: # stream tokens
        cache.append(token)
        while len(cache) >= L + 1:
            win, cache = cache[:L+1], cache[L+1:]
            inp, labels = win[:-1], win[1:]
            yield inp, labels

def get_batches(x, L, B):
    gen = chunk_generator(x, L)
    buf = []
    for inp, labels in gen:
        buf.append((inp, labels))
        if len(buf) == B:
            I = torch.tensor([p[0] for p in buf], dtype=torch.long)
            Y = torch.tensor([p[1] for p in buf], dtype=torch.long)
            yield I, Y
            buf = []
```

B Gradient Scan Procedure

After the batches are constructed, they are passed through the model to compute the loss. PyTorch's autograd engine then computes gradients for all parameters during back-propagation. To identify the most sensitive parameters, we record the maximum absolute gradient per element over many batches.

Algorithm B.1: Gradient Scan

Input: Model $f(\cdot;\vec{\theta})$, batches (I, Y), maximum steps S.
Output: For each parameter element θ_i, the maximum magnitude of the observed gradientradient.

1. Initialize a dictionary `running_max` mapping each parameter tensor to zeros of its shape.
2. For each batch (I, Y) up to S steps:
   (a) Zero model gradients.

(b) Compute the forward pass: $Z = f(I; \vec{\theta})$.

(c) Compute the loss: $\mathcal{L}(I, Y; \vec{\theta})$.

(d) Backpropagate: `loss.backward()`.

(e) For each parameter tensor p:

$$\texttt{running_max}[p] = \max\Big(\texttt{running_max}[p],\ |\nabla_p \mathcal{L}|\Big).$$

3. Return `running_max`.

Code Illustration

```
param_dict = {n: p for n, p in model.named_parameters() if p.requires_grad}
running_max = {n: torch.zeros_like(p, dtype=torch.float32) for n, p in
    param_dict.items()}

for step, (I, Y) in enumerate(batches, 1):
    model.zero_grad(set_to_none=True)
    loss = model(I, labels=I).loss # Hugging Face auto-aligns
    loss.backward()
    for name, p in param_dict.items():
        if p.grad is not None:
            running_max[name] = torch.maximum(running_max[name], p.grad.detach
                ().abs())
    if step >= MAX_STEPS:
        break
```

C Deepseek-R1-Distill-Qwen-1.5B Gradient Scan Results

The top-10 weights in the model (excluding embedding weights and layer-norm weights):

Local Rank	Global Rank	Weight	Element	$\lvert\nabla\mathcal{L}\rvert$
1	2	model.layers.3.mlp.gate_proj.weight	(8521, 713)	1.480×10^{0}
2	6	lm_head.weight	(1154, 507)	1.152×10^{0}
3	9	model.layers.3.mlp.gate_proj.weight	(8521, 872)	9.546×10^{-1}
4	10	lm_head.weight	(11, 507)	9.478×10^{-1}
5	11	model.layers.1.mlp.gate_proj.weight	(6988, 147)	9.453×10^{-1}
6	12	model.layers.0.self_attn.v_proj.weight	(98, 900)	9.424×10^{-1}
7	13	lm_head.weight	(1154, 1283)	9.375×10^{-1}
8	17	model.layers.3.mlp.gate_proj.weight	(8521, 520)	9.028×10^{-1}
9	19	lm_head.weight	(11, 1283)	8.203×10^{-1}
10	21	model.layers.0.mlp.down_proj.weight	(147, 2385)	8.120×10^{-1}